面向 Jetson 的分层记忆高斯子图在线重建系统报告

基于 **cuVSLAM** 前端、**HMR3D** 生命周期与 Active Gaussian Submap 的边缘化方案

CVPR Final Project Extended Report

2026-04-07

摘要

本报告在现有 **HMR3D** 方案基础上，进一步收敛并重写项目目标：不再将目标表述为“在任意硬件上做完整的全局在线 Gaussian SLAM”，而是提出一个更清晰、更可落地、也更适合课程项目与研究原型的系统命题：**在 Jetson 级边缘 GPU 上，实现一个受预算约束的长期在线三维重建系统，其中实时 tracking 由轻前端负责，长期记忆由分层记忆生命周期负责，而 Gaussian 只作为当前活动子图的高质量表示层。**

该系统的核心不是把 LoGeR、TTT3R、LoGoPlanner、Gaussian Splatting 和 Isaac 系列工程简单拼接起来，而是重新定义在线重建的长时程问题：单一 online state 无法同时承担局部高保真、长期保留、预算受限压缩和重访恢复四类职责，因此需要将系统升级为显式的 memory lifecycle。围绕这一判断，本报告提出 **HMR3D-GS Lite**：由 **cuVSLAM** 提供实时位姿与回环触发信号，由 **HMR3D** 管理 short-term / active / archive 三层记忆与 archive / retrieve / recover 生命周期，由 active Gaussian submap 负责局部高质量几何与外观表达，由 CPU / NVMe 侧 archive bank 保存冻结的历史子图，并在重访时按需恢复。

本报告首先界定这一方向与传统手持扫描、全局 GS-SLAM、纯 mesh 重建、纯 online recurrent reconstruction 的区别；随后系统梳理现有相关工作已经做到的部分与尚未系统化完成的部分；接着给出完整的方法定义、模块接口、Jetson 上的预算划分、实时性目标、实验协议、差异化叙事、风险控制与里程碑。报告的中心结论是：**本项目最稳的创新点不是“首次在 Jetson 上跑高斯”，也不是“首次把高斯用于 SLAM”，而是将 edge-oriented online 3D reconstruction 重构为“活动区域实时维护、历史区域归档保存、重访区域按需恢复”的分层记忆系统，并用 Gaussian submap 作为这一记忆系统的局部表达载体。**

目录

1 执行摘要 **5**

1.1 一句话定位 5

1.2 本报告想解决的不是哪个问题 5

1.3 最终希望达成的效果 5

2 问题定义与现实动机 **6**

2.1 为什么“手机已经能扫高斯”不等于这个项目没有意义 6

2.2 为什么“能实时跑高斯”很重要，但不必定义得过于极端 6

2.3 为什么单一 online state 不足以支撑 Jetson 上的长期高斯地图 7

3 现有工作做到了什么、没做到什么 **7**

3.1 四条已经被验证可行的技术线 7

3.1.1 第一类：轻前端 tracking 与可部署定位 7

3.1.2 第二类：纯在线 stateful reconstruction 7

3.1.3 第三类：在线 Gaussian SLAM 与高斯子图 8

3.1.4 第四类：边缘端资源约束与混合地图 8

3.2 你的方向与这些工作的真正区别 8

3.3 因此，最稳的 novelty 该怎么写 9

4 系统总览：HMR3D-GS Lite **9**

4.1 系统命题 9

4.2 高层模块图 9

4.3 核心设计原则 9

4.4 调度原则 10

5 表示与记忆：从 HMR3D 到 Active Gaussian Submap **10**

5.1 三层状态重新定义 10

5.2 活动高斯子图的数据结构 10

5.3 归档子图的数据结构 11

5.4 为什么 Gaussian 只做 active region 11

6 记忆生命周期：observe, promote, archive, retrieve, recover **12**

6.1 观察与活动化 12

6.2 晋升与锚定 12

6.3 归档 12

6.4 检索 13

6.5 恢复 13

7 Jetson 上的实时性与预算划分 **13**
7.1 为什么要先做系统预算，而不是先做 full GS 13
7.2 建议的预算分配 14
7.3 推荐的实时性目标 14
7.4 主线程与旁路线程 14
7.5 推荐输入模式 15
7.6 目标平台分层 15

8 为什么这不是简单拼装 **15**
8.1 表面上看像“把几个模块接起来” 15
8.2 真正的创新在于问题被重新定义了 16
8.3 更准确的项目陈述 16

9 项目可行性分析与验证闭环 **16**
9.1 为什么这件事在研究上可行 16
9.2 为什么这件事在工程上可行 17
9.3 验证不能只问“能不能跑”，而要问“哪一层假设被证实” 18
9.4 验证矩阵：每个命题对应什么证据 19
9.5 什么算完整成功，什么算部分成功 19

10 实验问题、评测协议与基线 **20**
10.1 三个核心研究问题 20
10.2 基线划分 20
10.3 评测协议 20
10.4 指标分组 21
10.5 关键消融 21

11 项目的学术意义与实际意义 **21**
11.1 学术意义 21
11.2 实际意义 22
11.3 对导航和语义的延展意义 22

12 风险、fallback 与工程策略 **22**
12.1 主要风险 22
12.2 fallback 路线 23
12.3 为什么这个 fallback 仍然有研究价值 23

13 实施路线与交付物 **23**
13.1 分层验收标准 23

13.2 阶段划分 24
13.3 接下来四周的具体计划 24
13.4 每阶段必须产出的可交付证据 25
13.5 建议交付物 25

14 结论 **25**

1 执行摘要

1.1 一句话定位

HMR3D-GS Lite 是一个**面向 Jetson 的分层记忆在线三维重建系统**：实时 tracking 交给 **cuVSLAM**，长期记忆组织交给 **HMR3D**，Gaussian 只维护当前 active region，历史区域以 archived submaps 形式保存，并在重访时 retrieve / recover。

1.2 本报告想解决的不是哪个问题

本项目不把自己定位成以下任一命题：

- 不是“第一次把 Gaussian Splatting 用于 SLAM”；
- 不是“第一次做高斯子图或 loop closure”；
- 不是“第一次把前端 tracking 和高斯后端解耦”；
- 也不是“在 Jetson 上复刻桌面级 full GS-SLAM”的纯工程移植。

这些方向中，已有工作分别验证了在线 Gaussian 建图、高斯子图、loop closure、前后端解耦、以及边缘端资源约束管理的可行性。真正尚未被系统化讲清楚、也是本项目最值得做的部分，是：

> **在受限显存、受限带宽、受限功耗的边缘平台上，如何把高斯地图从“持续全局维护对象”改造成“受记忆生命周期管理的局部活动表示”。**

1.3 最终希望达成的效果

本项目的最终效果分为三层：

1. **必须达成：** 在同一 backbone 或同一 tracking 前端下，仅通过更好的长期记忆组织，就能显著改善长程退化、重访恢复和分支消歧。
2. **系统达成：** archive / retrieve / recover 生命周期在线触发，而不是做成离线后处理。
3. **亮点达成：** 在 Jetson 上跑通一个 memory-bounded active Gaussian mapping demo，即 tracking 实时、active map 准实时、高斯只维护局部、历史区域按需恢复。

2 问题定义与现实动机

2.1 为什么“手机已经能扫高斯”不等于这个项目没有意义

近年来，手持设备和移动端确实已经可以完成一定形式的 3D 重建，甚至能在本地生成 Gaussian 表示。这说明一件事：**高斯表示本身并不再局限于桌面工作站。** 但这并不意味着边缘机器人上的长期在线建图问题已经被解决，因为两类问题的假设完全不同。

表 1: 移动端扫描与边缘机器人长期建图的差别

维度	手持 / 手机扫描	边缘机器人在线建图
输入形态	以一次性扫描任务为主，轨迹常由人控制	持续流式观测，轨迹受环境和任务驱动
目标	尽快出一个好看的模型或可视化结果	长期运行、长期维护、重访时仍可用
时间结构	常允许“扫完后再整理”	需要在线持续维护地图状态
资源结构	可容忍短时间峰值、离线整理或较慢后处理	必须长期受限于显存、带宽、功耗和散热
失败模式	局部缺面、细节不够、局部漂移	历史忘记、分支混淆、重访接不上、地图膨胀

因此，本项目真正研究的问题不是“高斯能不能在小设备上显示出来”，而是：

高斯地图能不能在小设备上长期活下来。

2.2 为什么“能实时跑高斯”很重要，但不必定义得过于极端

如果一点实时性都没有，只能离线整理，那么这个项目的工程意义会明显变弱；但如果把目标设成“全局地图每一帧都做完整高斯更新、全局渲染和全局优化”，项目又很容易在 Jetson 上直接失败。更合理的定义是分层实时：

- **tracking 实时：** 20–30 Hz 左右，至少要足以驱动机器人主循环；
- **active map 准实时：** 1–5 Hz 或 keyframe 级更新；
- **render 可降频：** 用于局部观察、调试或关键区域显示；
- **archive / recover 事件触发：** 不要求每帧执行，但必须在线可触发。

这一定义非常重要，因为它把“实时”从一个单一数字，改写成了**不同子系统的延迟预算分配问题**。这恰好与 **HMR3D** 的生命周期设计一致。

2.3 为什么单一 online state 不足以支撑 Jetson 上的长期高斯地图

无论是 recurrent state，还是全局 Gaussian map，只要把所有职责都压进一个持续更新对象里，都会遇到同一组结构性冲突：

1. **局部高保真** 需要保留最近观测的高分辨率信息；
2. **长期保持** 需要稳定、缓慢、可回忆的记忆结构；
3. **预算受限** 要求系统会主动遗忘和压缩；
4. **重访恢复** 要求历史信息不是被平均抹掉，而是能被检索、验证和注入。

对桌面 GPU 来说，常见做法是“多加显存、多留历史、多做一点优化”；但对 Jetson 来说，这种策略很快就会失效。于是问题自然转化为：

> **在线三维重建在边缘设备上的核心，不是继续维护一个越来越大的地图，而是设计一个会管理自身记忆生命周期的地图系统。**

3 现有工作做到了什么、没做到什么

3.1 四条已经被验证可行的技术线

基于目前已有工作与官方工程文档，可以把与你最相关的技术线概括为四类。

3.1.1 第一类：轻前端 tracking 与可部署定位

cuVSLAM 与 Isaac ROS Visual SLAM 已经证明，Jetson 上存在成熟、工程化、实时的视觉定位与回环基础设施。对本项目来说，这意味着最不应该重复发明的部分，就是前端 tracking。本项目的立场是：

> **tracking 不必由高斯反传承担；高斯表示的价值在于地图质量、长期记忆和下游可查询性，而不是强行取代成熟前端。**

3.1.2 第二类：纯在线 stateful reconstruction

TTT3R、LoGeR 等工作已经证明，在线 3D reconstruction 可以通过更聪明的状态更新和混合记忆获得更好的长序列表现。它们告诉我们两件事：

- 长程问题确实不只是 backbone 问题，而是状态维护问题；
- 但如果仍把长期信息压在一个主状态或一组连续传播状态里，archive / retrieve / recover 的能力仍然不够显式。

3.1.3 第三类：在线 Gaussian SLAM 与高斯子图

已有多类 Gaussian SLAM 工作验证了以下事情：

- 高斯表示可以服务于在线重建与 SLAM；
- 高斯子图、多子图、loop closure、submap fusion 是成立路线；
- 前端和高斯后端也可以松耦合；
- 大场景场景下，仅靠一张不断膨胀的全局高斯地图并不稳妥。

因此，本项目不能再把“高斯建图”或“高斯子图”本身包装成新颖点。

3.1.4 第四类：边缘端资源约束与混合地图

Isaac ROS nvblox 等系统表明，Jetson 上实时 mesh / TSDF 路线是现实可行的；相对地，原始 3D Gaussian Splatting 及其训练式变体对显存和持续优化成本更敏感。这形成了一个非常有价值的系统判断：

> **在边缘设备上，mesh 更适合作为稳定全局支撑层，Gaussian 更适合作为局部高质量活动表示层。**

3.2 你的方向与这些工作的真正区别

表 2 总结了现有工作与本项目之间最稳的差异化边界。

表 2: 已有方向与本项目的差异化边界

方向	已经做到了什么	本项目要补的缺口
轻前端 tracking	Jetson 上成熟的实时位姿估计、回环与地图复用	不重做 tracking，而是把它变成长期记忆系统的姿态与事件信号源
在线 stateful reconstruction	证明长序列确实受状态更新规则影响	将“状态更新”升级为显式 memory lifecycle，而非继续依赖单一主状态
Gaussian SLAM / Gaussian submaps	证明高斯可用于在线建图、submap、loop closure	将 Gaussian 从“全局持续优化对象”改写为“只维护 active region 的局部表示层”
Jetson / edge mapping	证明边缘端可做实时定位、mesh 重建和部分高斯维护	明确提出 active / archive / retrieve / recover 的 edge-oriented map lifecycle

3.3 因此，最稳的 novelty 该怎么写

最稳的表述不是“我们首次做到了某某”，而是：

> **现有工作已经分别证明了轻前端 tracking、在线 Gaussian 建图、高斯子图和边缘端滑窗管理各自可行；我们要解决的是，如何把它们统一到一个以长期记忆生命周期为中心、且适合 Jetson 预算约束的在线重建系统中。**

4 系统总览：HMR3D-GS Lite

4.1 系统命题

HMR3D-GS Lite 的主张可以压缩为一句话：

> **用 cuVSLAM 保持实时位姿主路径，用 HMR3D 管理长期记忆生命周期，用 Gaussian 只表达当前活动区域。**

4.2 高层模块图

```
Stereo/RGB-D/IMU
      |
      v
  cuVSLAM Frontend  ----->  poses / keyframes / loop signals
      |
      v
  HMR3D Memory Router
      |
      |---- M_short: recent observations / descriptors
      |---- M_active: active Gaussian submap on GPU
      |---- B_bank: archived submaps on CPU/NVMe
      |
      +---- retrieve / recover / merge on revisit
      |
      v
 local rendering / local map / optional mesh shadow map
```

4.3 核心设计原则

1. **tracking 与高斯解耦。** tracking 不依赖高斯反传优化，而由 **cuVSLAM** 保证实时主路径。

2. **活动区与历史区解耦。** 只有当前活动区域常驻 GPU；历史区域被显式归档。

3. **表示层与记忆层解耦。** **HMR3D** 决定什么该保留、归档、恢复；Gaussian 只负责“如何表达活跃局部地图”。

4. **全局支撑层与局部精细层解耦。** 允许在最终系统中同时保留一个稳定的 mesh / TSDF shadow map，避免把所有全局职责都压给 Gaussian。

4.4 调度原则

本系统还需要一个在答辩和实现中都必须保持一致的调度原则：

> **tracking 是实时主线，memory 是旁路系统；active Gaussian update、archive、retrieve 和 recover 只能消费 tracking 留下的预算，不能反过来阻塞 tracking。**

这句话的作用不是修辞，而是直接决定后续实现方式：如果某个模块会让 tracking 等待它完成，那么它就不应该被放在主路径上，而应被降到 keyframe 级、低频线程或事件触发线程中执行。

5 表示与记忆：从 HMR3D 到 Active Gaussian Submap

5.1 三层状态重新定义

在本项目中，**HMR3D** 的三层状态保留，但表示含义要重新写成更适合高斯系统的形式：

$$\mathcal{S}_t = \{M^t_{\text{short}},\ M^t_{\text{active}},\ B^t_{\text{bank}}\}. \tag{1}$$

- M^t_{short}：最近窗口的关键帧、局部描述子、短程观测 token、重访候选描述子。
- M^t_{active}：**当前活动高斯子图**，常驻 GPU，可执行局部增量更新与局部渲染。
- B^t_{bank}：冻结的历史子图库，常驻 CPU 或 NVMe，只保留检索、恢复和轻量校正所需表示。

5.2 活动高斯子图的数据结构

每个 active Gaussian submap 记为

$$G_t = \{\mu,\ R,\ s,\ \alpha,\ c,\ f,\ a,\ p,\ \eta\}, \tag{2}$$

其中：

- μ 为高斯中心；
- R 为旋转参数；
- s 为尺度；
- α 为 opacity；
- c 为颜色或低阶 SH 系数；
- f 为可选的压缩语义 / 外观特征；
- a 为 anchor 标记与重要性分数；
- p 为与当前活动子图坐标系相关的姿态锚定信息；
- η 为统计量，如观测次数、最近访问时间、可见性稳定度等。

5.3 归档子图的数据结构

每个归档子图记为

$$B_j = \{d_j,\ z_j,\ A_j,\ P_j,\ \Gamma_j,\ \Pi_j\}, \tag{3}$$

其中：

- d_j 为检索描述子；
- z_j 为 summary latent；
- A_j 为保留的关键 anchors 或压缩后的关键高斯；
- P_j 为子图参考位姿与局部坐标系；
- Γ_j 为轻量几何摘要，例如边界盒、稀疏关键点、统计尺度；
- Π_j 为元信息，包括时间段、重访次数、置信度和缓存位置。

5.4 为什么 Gaussian 只做 active region

这是本系统最关键的设计决定之一，原因有四：

1. active region 最需要高保真外观与几何一致表达；
2. 历史区域不需要持续占据 GPU，也不需要每帧参与 densify / prune；
3. Jetson 的瓶颈往往是持续更新成本，而不是偶尔恢复成本；
4. 这使得 archive / retrieve / recover 从“可选技巧”变成“系统基本机制”。

6 记忆生命周期：observe, promote, archive, retrieve, recover

6.1 观察与活动化

输入帧首先交给 **cuVSLAM**，得到位姿、回环触发信号与关键帧选择结果。随后系统把与当前 active submap 重叠较高的观测写入短程层，并在满足条件时对 active Gaussian submap 做局部更新。

局部更新不应等价于“每帧全量反传优化”。本报告建议的第一版实现是：

- 只在 keyframe 级执行高斯插入与轻量 refinement；
- 只更新 active submap 内可见区域；
- 对低贡献、高不确定性和低重访价值的高斯延迟 densify 或直接丢弃。

6.2 晋升与锚定

短程层中的局部观测不会无条件进入 active Gaussian submap，而应满足晋升条件。例如：

$$m_{\text{promote}}(i) = \mathbb{I}[s_{\text{imp}}(i) > \tau_s \wedge v(i) > \tau_v \wedge \rho(i) > \tau_r]. \tag{4}$$

这里 s_{imp} 为重要性，v 为可见性稳定度，ρ 为几何或重访价值。其本质是把 OVGGT 的 keep / evict 思路与 **HMR3D** 的 memory role 划分结合起来。

6.3 归档

当满足以下任一条件时，当前 active submap 被关闭并归档：

$$\mathcal{T}_{\text{archive}}(t) = \mathbb{I}[b_t > \tau_b \vee o_t < \tau_o \vee a_t > \tau_a \vee \ell_t = 1], \tag{5}$$

其中：

- b_t 为预算压力；
- o_t 为与当前区域的 overlap；
- a_t 为活动子图年龄；
- ℓ_t 为场景切换或回环边界事件。

归档时不保留整张 active Gaussian map 的全部自由度，而是执行：

1. 冻结不再需要实时更新的高斯；

2. 保留一小组 anchors 与稀疏关键高斯；

3. 生成 descriptor 与 summary latent；

4. 将大体量参数移出 GPU，转存 CPU / NVMe。

6.4 检索

检索不是普通 loop closure 的同义词，而是 **HMR3D** 生命周期中的一次**历史记忆召回**。系统使用当前描述子 d_t 在 archive bank 中检索 top-k 候选：

$$\mathcal{N}_k(t) = \underset{j \in B_{\text{bank}}}{\operatorname{argmax}} \cos(d_t, d_j). \tag{6}$$

为了降低重复走廊、相似房门、镜像空间等导致的假阳性，检索后必须再做几何验证：

$$\mathcal{T}_{\text{retrieve}}(t, j) = \mathbb{I}\left[\cos(d_t, d_j) > \tau_d \wedge \text{GeoVerify}(t, j) = 1\right]. \tag{7}$$

6.5 恢复

恢复不是把整个历史地图重新载入 GPU，而是：

1. 仅取回命中的 top-k archived submaps；

2. 仅恢复其 anchors、summary latent 和必要的局部高斯；

3. 将这些历史先验注入当前 active submap 的坐标系；

4. 仅对当前窗口做一次局部 refine；

5. 必要时再做轻量 merge，而不是全局重型 BA。

这一设计把“重访时如何找回历史”从一个后端专用问题，改写成**活动表示层与长期记忆层之间的信息注入问题**。

7 Jetson 上的实时性与预算划分

7.1 为什么要先做系统预算，而不是先做 full GS

在 Jetson 上，最容易失败的不是算法概念，而是资源分配失控。因此，本系统不以“全局 full GS-SLAM 能否运行”作为起点，而以“每个子模块最多占多少预算”作为起点。

7.2 建议的预算分配

表 3: Jetson 边缘版的预算分层建议

模块	推荐位置	预算与策略
cuVSLAM tracking	GPU + CPU	保持最高优先级，尽量实时，避免被高斯优化阻塞
短程描述子与事件检测	GPU / CPU 均可	尽量轻量化，支持重访候选与 archive 触发
active Gaussian submap	GPU 常驻	固定上限容量，局部更新、局部渲染、局部 densify
archived submap bank	CPU / NVMe	可增长，但每个条目都要压缩并带检索索引
retrieve / recover	事件触发	只在命中候选时搬回少量历史子图
全局 mesh shadow map	可选 GPU / CPU	用于稳定全局结构和导航安全，不要求与高斯同步高频更新

7.3 推荐的实时性目标

表 4: 分层实时性定义

层级	目标频率	说明
tracking	20–30 Hz	位姿主路径，不能明显掉速
keyframe selection	5–10 Hz	不必每帧建图，但要足够及时
active GS update	1–5 Hz	关键帧级增量更新即可
local render	2–10 Hz	调试或局部观察，不追求全图高帧率
archive / recover	event-driven	只在场景切换、重访、歧义时触发

7.4 主线程与旁路线程

为了让上面的预算表真正落地，本项目建议从一开始就按三类线程组织系统，而不是等所有模块写完后再做调度修补：

1. **Thread 1：实时主线程。** 只负责 **cuVSLAM** tracking、位姿更新、关键帧标记和 loop signal 产出，目标是稳定保持 20–30 Hz。

2. **Thread 2：记忆路由线程。** 负责 observe、promote、budget monitor、archive trigger 和 retrieve trigger。它可以消费关键帧，但不应阻塞主线程。

3. **Thread 3：低频映射线程。** 负责 active Gaussian submap 局部更新、recover 注入和必要的局部 merge，只在 keyframe 级或事件触发时运行。

这对应一个很关键的实现判断：observe 和 promote 可以靠近 tracking 放置，因为它们足够轻；archive 与 retrieve 优先在 CPU / NVMe 侧进行；recover 是最重的步骤，必须是 event-driven，并且只恢复少量命中子图。

7.5 推荐输入模式

在 Jetson 上，输入模式的选择会直接决定 active mapping 还有多少剩余预算。基于前述可行性分析，本报告建议：

1. **优先使用 Stereo 或 Stereo-Inertial。** 这类输入形态更适合作为系统默认配置，因为 tracking 负担更轻、延迟更稳定，也更容易把 GPU 预算留给 active mapping 与 recover。

2. **早期版本避免依赖额外的重型 Mono-Depth 前端。** 若前端本身已经消耗了较多 GPU，那么 active Gaussian patch、descriptor 检索和 recover 注入会更容易互相抢预算。

3. **把深度网络看作后续增强项，而不是一期前提。** 第一版原型应优先验证 lifecycle 是否成立，而不是把输入端复杂化。

7.6 目标平台分层

本项目建议把目标平台分成三档，而不是一开始就强行统一：

1. **Jetson Orin Nano 8GB：** 作为最小展示平台，目标是 tracking 实时、active GS 小规模运行、archive/recover 在线触发。

2. **Jetson Orin NX 16GB：** 作为更稳妥的系统原型平台，可容纳更大的 active submap 与更频繁的局部 refine。

3. **AGX Orin：** 作为上限参考平台，用于验证方案不是只在极端裁剪下才成立。

8 为什么这不是简单拼装

8.1 表面上看像“把几个模块接起来”

从模块来源看，**HMR3D-GS Lite** 的确借用了多条已有技术线：

- **cuVSLAM** 提供 tracking；

- **HMR3D** 提供分层记忆骨架；
- OVGGT / LoGoPlanner / MERG3R 的思想提供预算控制、query-based readout 与子图管理启发；
- Gaussian Splatting 提供局部高保真地图表示。

如果报告只把这些写成“用了 A、接了 B、加了 C”，那它会被很自然地理解成 assembly。

8.2 真正的创新在于问题被重新定义了

本项目的核心贡献不是“新模块表”，而是下列三件事情同时成立：

1. **问题定义改变。** 在线重建的长程失败模式被解释为 memory lifecycle 问题，而不是单一状态更新问题。
2. **职责划分改变。** tracking、active map、history archive、revisit recovery 的职责被明确解耦。
3. **成功标准改变。** 不再只看平均误差，而强调 fixed-budget horizon scaling、branch ambiguity、forced backtracking、revisit recovery。

8.3 更准确的项目陈述

因此，本项目应这样陈述自己：

> **我们不是在提出一个新的高斯表示，也不是简单把 LoGeR、TTT3R 和 Gaussian 拼起来；我们是在提出一种适用于 Jetson 级边缘平台的分层记忆地图系统，其中 Gaussian 只承担活动局部表达，而长期地图一致性由 archive / retrieve / recover 生命周期保障。**

9 项目可行性分析与验证闭环

9.1 为什么这件事在研究上可行

本项目之所以可行，不是因为已经存在一篇论文把整个系统原样做完，而是因为所需的关键能力已经被不同方向分别验证过。换句话说，我们不是在同时发明 tracking、在线重建、高斯建图和边缘部署，而是在把这些已经被验证可行的能力，重新组织为一个以 memory lifecycle 为核心的系统命题。

具体来说，至少有四条证据链支撑本项目的研究可行性：

1. **实时前端可行。** **cuVSLAM** 与 Isaac ROS 系列工作已经表明，Jetson 上存在成熟、工程化、可部署的实时视觉定位与回环基础设施，因此本项目没有必要把最脆弱的 tracking 主路径也一起重做。

2. **长程在线重建确实受记忆组织影响。** TTT3R、LoGeR 一类工作已经说明，长序列退化并不只是 backbone 表达能力不够，更与 online state 如何更新、压缩和保留有关。

3. **Gaussian submap 路线可行。** 已有 Gaussian SLAM、多子图与 loop closure 工作已经证明，高斯表示并不只能用于离线训练后的新视角合成，它也可以服务于在线建图、局部地图维护和子图级别的融合。

4. **边缘端预算约束建图可行。** nvblox 与一系列 Jetson 端 mapping 系统已经证明，只要把表示和更新频率设计对，边缘设备并不是不能建图，而是不能无约束地维护一个无限膨胀的全局对象。

因此，本项目真正待验证的不是“这些模块各自能不能存在”，而是更明确的一条系统性假设：

> **在边缘预算约束下，把地图写成 active / archive / retrieve / recover 生命周期，比持续维护一个单一全局在线状态更适合长期在线三维重建。**

9.2 为什么这件事在工程上可行

工程上最关键的判断是：**并不是所有模块都必须以同样频率、同样预算持续运行。**一旦接受分层实时性的定义，系统成本就可以被写成一个事件驱动的预算闭环，而不是“每一帧都做全量高斯更新”。记总成本为

$$C_t = C^t_{\text{track}} + \mathbb{I}_{k_t} C^t_{\text{active}} + \mathbb{I}_{r_t} C^t_{\text{render}} + \mathbb{I}_{e_t} C^t_{\text{archive/recover}}, \tag{8}$$

其中 k_t 表示当前时刻是否触发 keyframe 级 active map 更新，r_t 表示是否触发局部渲染，e_t 表示是否触发 archive 或 revisit recover 事件。这个表达式的含义很直接：**只有 tracking 是每帧硬约束，地图维护、渲染和恢复都可以按事件或按关键帧执行。**

围绕这一定义，本项目的工程可行性来自五个具体决策：

1. **固定 tracking 主路径。** 让 **cuVSLAM** 负责位姿主路径，避免把“实时定位是否稳定”和“高斯地图是否精细”耦合成同一个风险点。

2. **限制 Gaussian 的职责边界。** Gaussian 只维护当前 active region，而不是常驻全局地图；这样 GPU 上被持续更新的对象规模是可控的。

3. **把历史转移到 CPU / NVMe。** archive bank 不追求随时可渲染，只要求可检索、可恢复、可做轻量几何校验，因此完全可以放到 GPU 之外。

4. **把重操作改成事件触发。** archive、retrieve、recover 不要求每帧运行，只在预算压力、区域切换、重访候选或分支歧义时触发。

5. **从一开始保留退化路径。** 若 active Gaussian 仍然偏重，可退化为 mesh shadow map + tiny active GS patch；若 recover 误触发严重，可先做 descriptor recall，再做 geometry verification。

这意味着本项目不是在押注“Jetson 能否暴力跑完整 full GS-SLAM”，而是在构造一个**预算上闭合、职责上分层、故障时可退化**的系统。

9.3 验证不能只问“能不能跑”，而要问“哪一层假设被证实”

对这类系统项目而言，最常见的问题是把“demo 跑起来了”误当成“研究命题被证实了”。本项目需要避免这种叙事错误，因此验证必须分层进行，而不是把所有目标揉成一个二元判断。

本项目至少需要验证三层假设：

1. **记忆假设。** 在相同 tracking 前端和近似相同预算下，active / archive / retrieve / recover 的生命周期设计，是否真的优于单一不断膨胀的 active map。

2. **表示假设。** 在生命周期设计已经成立的前提下，active Gaussian submap 是否比 mesh-only active map 提供更好的局部质量、局部可视化或局部可查询性。

3. **边缘部署假设。** 上述收益能否在 Jetson 级平台上以可接受的频率和资源占用在线触发，而不是只在桌面 GPU 上成立。

这种分层验证很重要，因为它决定了结果的解释方式。若第一层成立而第三层只做到最小展示版，那么研究结论仍然部分成立，只是工程落地尚未完成；反过来，若 Jetson 上 demo 能跑但第一层没有带来长期记忆收益，那么项目的核心命题仍然没有被证实。

9.4 验证矩阵：每个命题对应什么证据

表 5: 项目可行性与验证矩阵

待验证命题	正向证据	若不成立的信号
memory lifecycle 优于单一 active state	在固定预算下，horizon scaling 更慢退化，branch ambiguity 更少，long-gap revisit 恢复更稳定	长序列误差与无 archive 基线接近，或 bank 增长很快但几乎没有带来恢复收益
retrieve / recover 机制有用	top-k retrieval recall 稳定，几何验证后错误恢复率可控，恢复后局部误差和全局一致性同时改善	检索假阳性高、恢复后误差反而上升，或 recover 触发频繁但没有实际收益
active Gaussian 值得其成本	在相近预算下，相比 mesh-only active map 获得更好的局部几何/外观质量，或更强的局部渲染与查询能力	运行代价明显更高，但局部质量、可视化或下游接口没有显著收益
Jetson 上的在线触发成立	tracking 保持实时，active map 达到 keyframe 级更新，archive / recover 能在线触发且不会造成持续失速	只能离线处理 archive/recover，或一旦开启 active GS / recover 就导致端到端系统不可持续运行

9.5 什么算完整成功，什么算部分成功

为了避免最终结论含糊，本项目可以把结果解释分成三档：

1. **完整成功。** 在 Jetson 或至少 Orin 级目标平台上，**HMR3D-GS Lite** 完整跑通，并且在固定预算下相对基线显著改善长程退化、分支消歧或重访恢复。

2. **部分成功。** 在 x86 / 桌面平台上清楚验证了 memory lifecycle 的收益，同时在 Jetson 上跑通最小展示版，证明 active / archive / recover 的在线触发链路成立。

3. **负结果但仍有价值。** 若 active Gaussian 的收益不足以覆盖成本，但 memory lifecycle 本身被证实有效，那么结论应当收缩为“边缘长期在线重建的关键收益来自记忆生命周期设计，而非必须来自 Gaussian 本身”。

相应地，真正会削弱本项目核心命题的结果也应被明确写出：如果在相同 tracking 前端和相近预算下，引入 archive / retrieve / recover 后并未改善长期失败模式，或者 recover 的假阳性使系统整体变差，那么就不能再声称 memory lifecycle 是这里的关键贡献。这种负判据必须提前写进报告，才能让“可行性分析与验证”具有研究上的约束力，而不是事后解释。

10 实验问题、评测协议与基线

10.1 三个核心研究问题

1. **RQ1：** 在固定显存预算下，active / archive 解耦是否优于单一不断膨胀的活动地图？

2. **RQ2：** retrieve / recover 是否真的能降低 long-gap revisit 和 branch confusion 的失败率？

3. **RQ3：** 在 Jetson 上，只维护 active Gaussian submap 是否能在可接受实时性下带来比 mesh-only 更好的局部质量或下游可查询性？

10.2 基线划分

建议至少准备四组基线：

1. **Tracking / mesh 基线：** **cuVSLAM** + nvblox 类系统，不含高斯，不含长期 archive/recover。

2. **无 archive 的 active GS 基线：** 前端一致，只维护一个滑动窗口 Gaussian submap。

3. **无 retrieve 的 archive 基线：** 有归档，但不做恢复，仅做压缩。

4. **HMR3D-GS Lite 完整系统：** active Gaussian + archive bank + retrieve / recover。

10.3 评测协议

除了常规 pose / depth / reconstruction 指标，更重要的是以下四组 stress tests：

1. **Horizon Scaling：** 随序列长度增长，误差与延迟如何退化。

2. **Branch Disambiguation：** 重复结构下是否更容易发生身份混淆。

3. **Forced Backtracking：** 远离后再返回时，是否仍能利用历史。

4. **Revisit Recovery：** retrieve / recover 触发后，误差是否真正下降。

10.4 指标分组

表 6: 指标分组

指标组	代表指标
Tracking	ATE、RPE、tracking 丢失率、回环后位姿修正幅度
Mapping quality	局部渲染误差、深度误差、点云精度/完整度、局部几何一致性
Memory behavior	active 高斯数量、archive bank 大小、top-k retrieval recall、错误恢复率
Edge efficiency	FPS、端到端延迟、GPU 占用、CPU 占用、峰值显存、功耗

10.5 关键消融

必须做的消融包括：

1. 去掉 archive，只保留 active GS；
2. 去掉 retrieve/recover，只做 archive；
3. 去掉重要性裁剪，只保留固定容量 FIFO；
4. 去掉 anchors，只保留普通高斯；
5. 用简单平均池化描述子替代 query-based summary；
6. 用 mesh-only active map 替代 Gaussian active map。

11 项目的学术意义与实际意义

11.1 学术意义

本项目最主要的学术意义，不在于提出一个全新 backbone，而在于推动以下判断更明确地成立：

在线三维重建真正缺的不是再多一个表示头，而是长期记忆组织能力。

更具体地说，它的贡献可以写成三条：

1. 将长时程在线重建从“单一 state update”重写为“memory lifecycle design”问题；
2. 给出一种边缘预算约束下的 active / archive / retrieve / recover 地图系统；
3. 建立比普通 pose / depth 指标更贴近长期记忆失败模式的 stress-test 评测协议。

11.2 实际意义

本项目真正的工程价值，不是让 Jetson 上“也能看到高斯”，而是把长期在线地图从

持续全量维护

改造成

局部活动维护 + 历史归档保存 + 需要时再恢复。

这对于实际机器人意味着：

- 地图不再随着时间无限膨胀；
- 历史区域不再只是负担，而是可检索资源；
- 高斯表示不再只是视觉展示，而可以成为局部决策、局部语义、局部导航的高质量世界模型。

11.3 对导航和语义的延展意义

若后续系统稳定，本方向还自然支持两类扩展：

1. **语义扩展：** 只在 active Gaussian submap 中维护压缩语义特征，再随 archive 一并存储，形成可重访的 3D 语义局部地图。
2. **导航扩展：** 将 active Gaussian submap 作为局部感知世界模型，服务于 near-field navigation 或 semantic goal-conditioned behavior。

因此，实时高斯的意义不只是“画面更好看”，而是让地图从几何缓存升级为可渲染、可查询、可检索的活动记忆。

12 风险、fallback 与工程策略

12.1 主要风险

1. **风险 1：** Jetson 上高斯增量更新仍然过重。
2. **风险 2：** retrieve 误触发导致错误恢复。
3. **风险 3：** archive 设计不当，bank 越积越大但并不有用。
4. **风险 4：** AArch64 上高斯相关依赖链、编译链或 CUDA 内核不稳定。

12.2 fallback 路线

- 若 active Gaussian 仍然过重，则退化为 **mesh shadow map + tiny active GS patch**；
- 若 retrieve 假阳性严重，则引入更严格的两段式验证：descriptor recall + geometry verification；
- 若 archive bank 失控，则增加 bank budget 与二级压缩；
- 若 Jetson 编译链不稳，则先在 x86 桌面验证完整生命周期，再迁移到 Jetson 只保留最小功能版。

12.3 为什么这个 fallback 仍然有研究价值

即使最后没有做成“完整 active Gaussian mapping”，本项目仍然有价值，只要以下命题能被证实：

> **长期在线重建在边缘设备上的核心收益，来自记忆生命周期设计，而不是来自某一种单独表示。**

这意味着即便最终落地为 mesh + archived descriptors + partial GS patch，本项目仍然成立。

13 实施路线与交付物

13.1 分层验收标准

为了避免项目推进到最后才发现成功标准过于含糊，本报告建议把验收标准分成三层：

1. **第一层：P0/P1 验收。** **cuVSLAM** 数据流、关键帧缓存、short / active / archive 三层状态以及 archive / retrieve / recover 路由跑通。此时允许 active map 还是 mesh-only 或 dummy GS。
2. **第二层：P2 验收。** active Gaussian patch 可以局部更新、局部渲染、局部恢复，并且不会拖垮 tracking 主路径。
3. **第三层：P3/P4 验收。** 在固定预算下，相比 no-archive 或 no-retrieve 基线，在 long-gap revisit、branch ambiguity、forced backtracking 上具有更低失败率或更稳定退化曲线。

这意味着本项目的最小成功标准并不是“Jetson 上出现一张漂亮完整的高斯地图”，而是：**memory lifecycle 在边缘预算下被证明确实有用，Gaussian 作为 active local world model 带来了可感知的局部收益。**

13.2 阶段划分

1. **P0：稳定基线。** 打通 **cuVSLAM** + 数据流 + 关键帧缓存 + 最基础局部地图接口。
2. **P1：HMR3D 生命周期。** 实现 short / active / archive 三层状态，以及 archive / retrieve / recover 路由。
3. **P2：Active Gaussian。** 只在 active region 引入小规模高斯表示与局部更新。
4. **P3：Jetson 裁剪。** 固定容量、降频渲染、低阶表示、CPU/NVMe archive。
5. **P4：重访恢复与对比实验。** 做完整 stress tests 与系统对比。

13.3 接下来四周的具体计划

如果目标是尽快得到可验证结果，而不是先追求完整系统外观，那么最现实的四周计划应当是：

1. **第 1 周：打通实时主路径。** 固定 Stereo 或 Stereo-Inertial 输入；完成 **cuVSLAM** 位姿、keyframe、loop signal 的统一输出接口；记录端到端 latency、GPU 占用和关键帧缓存行为。
2. **第 2 周：落地 lifecycle 骨架。** 实现 `M_short / M_active / B_bank` 的结构化状态对象；先用轻量 descriptors 跑通 observe、promote、archive 和 retrieve trigger，不急着接入完整高斯更新。
3. **第 3 周：接入 tiny active Gaussian patch。** 只在 active region 做 keyframe 级更新；把 recover 限制为 top-k 命中子图 + 局部 refine；同时验证 tracking 是否仍保持实时主路径。
4. **第 4 周：做最小闭环实验。** 至少跑通 no-archive、archive-only、full lifecycle 三组对比；优先看 revisit recovery、branch ambiguity 和 fixed-budget horizon scaling，而不是先追求全量美观渲染。

这四周计划的关键原则是：**先验证 lifecycle 的存在价值，再扩大 Gaussian 的覆盖范围。** 如果顺序反过来，项目很容易陷入“局部地图越来越重，但还没有证明记忆设计是否有效”的状态。

13.4 每阶段必须产出的可交付证据

每个阶段都应输出一个可以复核的证据，而不是只输出“代码已经写了”：

1. **P0**： tracking 频率、关键帧频率、Jetson 资源占用表。
2. **P1**： archive bank 样例、retrieve 命中日志、recover 触发日志。
3. **P2**： active Gaussian patch 的局部更新视频或局部渲染结果。
4. **P3**： 固定 budget 下 active 高斯数量、bank 大小与延迟曲线。
5. **P4**： stress tests 和基线对比图表。

13.5 建议交付物

最终建议交付以下内容：

1. 一份桌面端可运行的 **HMR3D-GS Lite** 原型；
2. 一份 Jetson 上的最小展示版；
3. 一套以 horizon scaling、branch ambiguity、revisit recovery 为核心的实验结果；
4. 一份系统性报告，明确说明哪些增益来自 memory lifecycle，哪些来自 Gaussian representation。

14 结论

本报告基于现有 **HMR3D** 思路与前述调研，提出了一个更聚焦、也更有机会真正落地的项目版本：**HMR3D-GS Lite**。它不把目标设成“在 Jetson 上复刻完整 full GS-SLAM”，而是把问题收缩为一个更强、更清晰的系统命题：**在固定预算下，让高斯地图只承担当前活动区域的局部表达，而把长期一致性、重访恢复和历史组织交给显式记忆生命周期。**

从项目叙事上看，这一版本的最大优势在于：

- 它不和已有“在线 Gaussian SLAM 已经存在”这一事实正面冲突；
- 它不把自己包装成不现实的“Jetson 上全局全时高斯训练”；
- 它把最核心的创新点放在长期记忆组织与 edge-oriented system design 上；
- 它允许 Gaussian 成为亮点，但不让 Gaussian 抢走项目真正的问题意识。

因此，这个项目最值得坚持的最终表述应当是：

我们要做的不是“Jetson 上也能跑高斯”，而是“让边缘设备上的在线三维地图第一次具备可持续的记忆生命周期，而 Gaussian 只是当前活动世界的高质量表达层”。

参考资料

1. NVIDIA Isaac ROS Visual SLAM 文档：https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_visual_slam/index.html
2. NVIDIA Isaac ROS nvblox 文档：https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_nvblox/index.html
3. NVIDIA Isaac cuVSLAM 官方仓库：https://github.com/nvidia-isaac/cuVSLAM
4. TTT3R 官方仓库：https://github.com/Inception3D/TTT3R
5. CUT3R 官方仓库：https://github.com/CUT3R/CUT3R
6. 原始 3D Gaussian Splatting 官方仓库：https://github.com/graphdeco-inria/gaussian-splatting
7. gsplat 官方仓库：https://github.com/nerfstudio-project/gsplat